



Random-Key Genetic Algorithms: Principles and Applications

30

Mariana A. Londe , Luciana S. Pessoa , Carlos E. Andrade ,
José Fernando Gonçalves , and Mauricio G. C. Resende

Contents

Introduction.....	922
Biased Random-Key Genetic Algorithms.....	923
A Model for the Implementation of a BRKGA.....	924
Optional Procedures of the Framework.....	926
Hybridizing a Random-Key Genetic Algorithm.....	928
Customizing a RKGA.....	928
Tuning an RKGA.....	929
APIs for BRKGA.....	931
Conclusion.....	932
Cross-References.....	932
References.....	933

Abstract

A random-key genetic algorithm is an evolutionary metaheuristic for discrete and global optimization. Each solution is encoded as a vector of n random keys, where a random key is a real number randomly generated in the continuous

M. A. Londe · L. S. Pessoa

Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

e-mail: mlonde@aluno.puc-rio.br; lucianapessoa@puc-rio.br

C. E. Andrade

AT&T Labs Research, Bedminster, NJ, USA

e-mail: cea@research.att.com

J. F. Gonçalves

Faculdade de Economia da, Universidade do Porto, Porto, Portugal

M. G. C. Resende (✉)

Industrial and Systems Engineering, University of Washington, Seattle, WA, USA

e-mail: mgcr@berkeley.edu

interval $[0, 1)$. A decoder maps each vector of random keys to a solution of the optimization problem being solved and computes its cost. The benefit of this approach is that all genetic operators and transformations can be maintained within the unitary hypercube, regardless of the problem being addressed. This enhances the productivity and maintainability of the core framework. The algorithm starts with a population of p vectors of random keys. At each iteration, the vectors are partitioned into two sets: a smaller set of high-valued elite solutions and the remaining non-elite solutions. All elite elements are copied, without change, to the next population. A small number of random-key vectors (the mutants) are added to the population of the next iteration. The remaining elements of the population of the next iteration are generated by combining, with the parametrized uniform crossover of Spears and DeJong (On the virtues of parameterized uniform crossover. In: Proceedings of the Fourth International Conference on Genetic Algorithms, pp 230–236, 1991), pairs of solutions. This chapter reviews random-key genetic algorithms and describes an effective variant called biased random-key genetic algorithms.

Keywords

Genetic algorithm · Random keys · Optimization · Random-key genetic algorithm · Biased random-key genetic algorithm · Metaheuristic · Parameter tuning

Introduction

Genetic algorithms (GAs) are inspired by Darwin's theory of evolution. Holland [39] introduces this approach, which considers that a possible solution to a combinatorial optimization problem is an individual inside a population of potential solutions. In this population, evolutionary operators are applied at each iteration/generation to obtain new populations and solutions.

A class of GAs is described by Bean [9], whose solutions are represented by a vector of random keys, called a chromosome. A *random key* is a real number belonging to the interval $[0, 1)$. This makes the evolutionary process of the *random-key genetic algorithms* (RKGAs) problem-independent, as the evolutionary operators can be specified for the chromosomes, which allows for a generalization of the framework. The quality of a solution is calculated by the *decoder* procedure, which maps a chromosome to a fitness value. In [9], the decoding procedure simply orders the elements of the vector of random keys, which produces a permutation of the indices of the sorted elements.

In the RKA evolutionary process, the algorithm starts with an initial population $\mathcal{P}$ of vectors of n random keys. In each generation, the vectors are partitioned into a set of the best solutions $\mathcal{P}^e < |\mathcal{P}|/2$, called the elite set, and a set with the remainder of non-elite solutions of the population. All elite solutions are copied, unchanged, into the population of the next generation. This elitism characterizes the Darwinian

principle in the RKGA. Next, a set $\mathcal{P}^m$ of randomly created vectors is introduced into the next population. This set comprises *mutants*, which play the same role as the mutation operators of classical GAs, i.e., they help increase population diversity and avoid convergence to non-global local optimum. To complete the elements of the next population, $|\mathcal{P}| - |\mathcal{P}^e| - |\mathcal{P}^m|$ are generated by combining solutions from the entire population using a parametrized uniform crossover [107, 108]. In this crossover, consider a and b to be the vectors chosen for mating, and let c be the offspring produced. Random key $c[i]$, the i -th component of the offspring vector, receives the i -th key of one of its parents. It receives the key $a[i]$ with probability ρ_a and $b[i]$ with probability $\rho_b = 1 - \rho_a$.

Biased Random-Key Genetic Algorithms

A *biased random-key genetic algorithm* (BRKGA), meanwhile, also uses an elitist strategy in the mating procedure, not only regarding the reproduction of the elite set.

In BRKGA, one parent is always chosen from the elite set. When performing the coin toss to choose whose parent passes the gene to the offspring, the gene of the elite parent has a probability of being inherited $\rho > 0.5$. This means that the offspring has a higher chance of inheriting the keys of the elite parent and of retaining substructures of a good solution while still allowing the insertion of substructures of a not-so-good solution. This difference between the two algorithms tends to result in BRKGA outperforming Bean's algorithm [38]. As a result, most of the literature published after the introduction of BRKGA tends to use the biased variant. A flowchart detailing the BRKGA framework can be seen in Fig. 1. In it, the problem-independent structure of classical BRKGA is highlighted. The decoding procedure in black is problem-dependent, as it makes the correlation between a random-key vector and a feasible problem solution.

The double elitism of BRKGA leads to rapid convergence to high-quality solutions. Alongside its problem-agnostic approach, it has led to a significant increase in use, with over 250 papers published until 2025 [56, 57]. The BRKGA was formally introduced in Gonçalves and Resende [36], but some of its characteristics can be seen in the early 2000s [15, 27, 35]. Since then, several applications, hybridizations, and features have been described in the literature. Among the problems in which BRKGA was applied since 2020, the following are considered:

- *Scheduling*: Abreu et al. [1], Abreu et al. [2], Bolsi et al. [11], Fontes and Homayouni [31], Fontes et al. [32], Huang et al. [42], Ibarra-Rojas et al. [44], Kong et al. [48], Kong et al. [49], Kummer et al. [50], Liu et al. [52], Maecker et al. [61], Queiroga et al. [87], Rocholl and Mönch [94], Silva-Soto and Ibarra-Rojas [103], Soares and Carvalho [105], Soares and Carvalho [104], Xie et al. [116], Yang et al. [118], Yu et al. [119], Zhang et al. [120], and Zhao et al. [121]
- *Network configuration*: Mikulski and Tomczewski [68], Pinto et al. [85], Raposo et al. [89], and Usberti et al. [111]

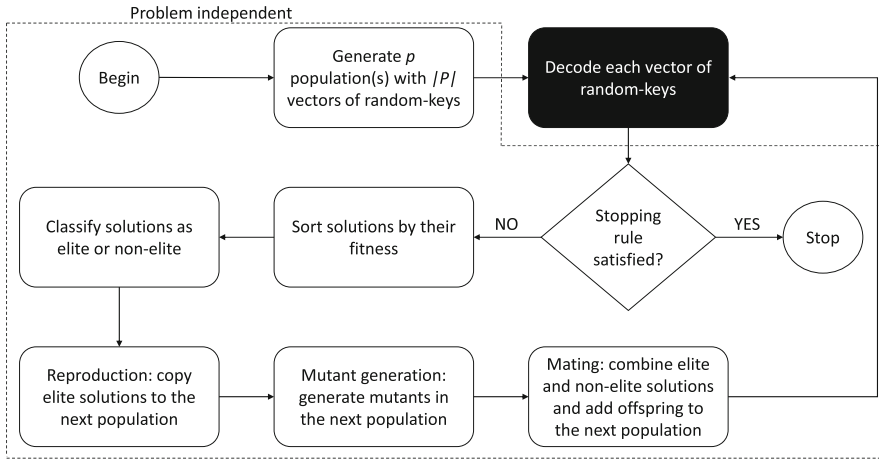


Fig. 1 Flowchart detailing BRKGA process (Adapted from Gonçalves and Resende [36])

- *Location*: Freitas et al. [33], Johnson et al. [46], Londe et al. [53], Mauri et al. [65], Mendoza-Andrade et al. [67], and Villicaña-Cervantes and Ibarra-Rojas [113]
- *Cutting and packing*: Amaro Júnior et al. [3], Gonçalves and Wäscher [37], Lu et al. [60], Mandal et al. [62], Oliveira et al. [75], Oliveira and Romero [71], Souza Queiroz and Andretta [106], and Zhou et al. [122]
- *Traveling salesman and vehicle routing*: Amaro Júnior et al. [4], Carrabs [16], Chagas et al. [17], Chagas et al. [18], Chaves et al. [21], Damm et al. [25], Huang et al. [40], Huang et al. [41], Ibarra-Rojas and Silva-Soto [43], Marques Jr et al. [63], Oliveira et al. [77], Santiyuda et al. [95], Schenekemberg et al. [96], Schenekemberg et al. [97], and Schuetz et al. [98]
- *Clustering*: Brito et al. [13], Fadel et al. [28], Martarelli and Nagano [64], and Xu et al. [117]
- *Graph problems*: Lima et al. [51], Londe et al. [54], Londe et al. [55], Melo et al. [66], Pinto et al. [86], Silva et al. [102], and Silva et al. [101]
- *Automatic tuning of parameters in heuristics*: Cicek and Ozturk [22], Falls et al. [29], Japa et al. [45], and Sun and Zhou [109]
- *Other applications*: Morgan et al. [69], Ochoa et al. [70], Pan et al. [79], Pastore et al. [81], Pinacho-Davidson and Blum [84], Ramos et al. [88], Reixach et al. [90], and Silva et al. [100]

A Model for the Implementation of a BRKGA

Algorithm 1 shows a pseudo-code of a BRKGA for the minimization of $f(x)$, where $x \in X$ and X is a discrete set of solutions and $f : X \rightarrow \mathbb{R}$. This implementation is a multi-start variant of a BRKGA where several populations are evolved in sequence,

Algorithm 1: Model for biased random-key genetic algorithm with restart

```

1 BRKGA( $|\mathcal{P}|$ ,  $|\mathcal{P}_e|$ ,  $|\mathcal{P}_m|$ ,  $n$ ,  $\rho_a$ )
2 Initialize value of the best solution found:  $f^* \leftarrow \infty$ ;
3 while stopping criterion not satisfied do
4   Generate a population  $\mathcal{P}$  with  $|\mathcal{P}|$  vectors of  $n$  random keys;
5   while restart criterion not satisfied do
6     Evaluate the cost of each new solution in  $\mathcal{P}$ ;
7     Partition  $\mathcal{P}$  into two sets:  $\mathcal{P}_e$  and  $\mathcal{P}_e^-$ ;
8     Initialize population of next generation:  $\mathcal{P}^+ \leftarrow \mathcal{P}_e$ ;
9     Generate set  $\mathcal{P}_m$  of mutants, each mutant with  $n$  random keys;
10    Add  $\mathcal{P}_m$  to population of next generation:  $\mathcal{P}^+ \leftarrow \mathcal{P}^+ \cup \mathcal{P}_m$ ;
11    foreach  $i \leftarrow 1$  to  $|\mathcal{P}| - |\mathcal{P}_e| - |\mathcal{P}_m|$  do
12      Select parent  $a$  at random from  $\mathcal{P}_e$ ;
13      Select parent  $b$  at random from  $\mathcal{P}_e^-$ ;
14      foreach  $j \leftarrow 1$  to  $n$  do
15        Throw a biased coin with probability  $\rho > 0.5$  of resulting heads;
16        if heads then  $c[j] \leftarrow a[j]$ ;
17        else  $c[j] \leftarrow b[j]$ ;
18      end
19      Add offspring  $c$  to population of next generation:  $\mathcal{P}^+ \leftarrow \mathcal{P}^+ \cup \{c\}$ ;
20    end
21    Update population:  $\mathcal{P} \leftarrow \mathcal{P}^+$ ;
22    Find best solution  $\chi^+$  in  $\mathcal{P}$ :  $\chi^+ \leftarrow \text{argmin}\{f(\chi) \mid \chi \in \mathcal{P}\}$ ;
23    if  $f(\chi^+) < f^*$  then
24       $\chi^* \leftarrow \chi^+$ ;
25       $f^* \leftarrow f(\chi^+)$ ;
26    end
27  end
28 end
29 return  $\chi^*$ 

```

and the best solution among all in the population is returned as the output of the algorithm.

In line 2, the value f^* of the best solution found is initialized to a large value, i.e., not smaller than $f(x^0)$, where $x^0 \in X$ is some feasible solution to the problem. Evolution of each population is done in lines 3–28. The algorithm halts when a stopping criterion in line 3 is satisfied. This criterion can be, for example, the number of evolved populations, the total time, or the quality of the best solution found.

In line 4, the population being evolved is initialized with $|\mathcal{P}|$ vectors of random keys. Evolution of population $\mathcal{P}$ takes place in lines 5–27. This evolution ends when a restart criterion is satisfied in line 5. This criterion can be, for example, a maximum number of generations without improvement in the value of the best solution in $\mathcal{P}$. At each generation, or iteration, the following operations are carried out: In line 6, all new solutions (offspring and mutants) are decoded and their costs evaluated. Note that each decoding and evaluation in this step can be computed simultaneously, i.e., in parallel. In line 7, population $\mathcal{P}$ is partitioned into two subpopulations $\mathcal{P}_e$

(elite) and $\mathcal{P}_e$ (non-elite), where $\mathcal{P}_e$ is such that $|\mathcal{P}_e| < |\mathcal{P}|/2$ and contains $|\mathcal{P}_e|$ of the best solutions in $\mathcal{P}$ and $\mathcal{P}_e$ consists of the remaining solutions in $\mathcal{P}$, that is, $\mathcal{P}_e = \mathcal{P} \setminus \mathcal{P}_e$. $\mathcal{P}^+$ is the population of the next generation. It is initialized in line 8 with the elite solutions of the current generation. In line 9, the mutant subpopulation $\mathcal{P}_m$ is generated. Each mutant is a vector of n random keys. The number of generated mutants in general is such that $|\mathcal{P}_m| < |\mathcal{P}|/2$. This subpopulation is added to the population $\mathcal{P}^+$ of the next generation in line 10.

With $|\mathcal{P}_e| + |\mathcal{P}_m|$ vectors inserted in population $\mathcal{P}^+$, it is necessary to generate $|\mathcal{P}| - |\mathcal{P}_e| - |\mathcal{P}_m|$ new offspring to complete the $|\mathcal{P}|$ vectors that form population $\mathcal{P}^+$. This is done in lines 11–20. In lines 12 and 13 parents a and b are chosen, respectively, at random from subpopulations $\mathcal{P}_e$ and $\mathcal{P}_e$. The generation of offspring c from parents a and b occurs in lines 14–18. A biased coin (with probability $\rho > 0.5$ of flipping to heads) is thrown n times. If the i -th toss is a head, the offspring inherits the i -th key of parent a . Otherwise, it inherits the i -th key of parent b . After the offspring is generated, c is added to population $\mathcal{P}^+$ in line 19.

The generation of $\mathcal{P}^+$ ends when it consists of $|\mathcal{P}|$ elements. In line 21, $\mathcal{P}^+$ is copied to $\mathcal{P}$ to start a new generation. The best solution in the current population in evolution is computed in line 22, and if its value is better than all solutions examined so far, the solution and its cost are saved in lines 24 and 25 as χ^* and f^* , respectively. χ^* , the best solution found across all populations, is returned by the algorithm in line 29.

Optional Procedures of the Framework

There are several ways to modify the BRKGA algorithm in order to increase solution quality, convergence, diversity, and other criteria. One of these is the use of the restart. In BRKGA, the restart or reset of the population is the elimination of all current individuals and the recreation of the entire population with random chromosomes. This is done, generally, after I_r number of iterations without improvement in the best overall solution χ^* , similar to the restart strategy for GRASP with path-relinking [91], has been shown to increase individual diversity and prevent premature convergence [80, 110].

A partial restart is also possible. The shaking method [6] is akin to a perturbation of the Iterated Local Search (ILS) algorithm [59] and acts by modifying a part of the genes of the elite solutions, while the non-elite chromosomes are completely restarted. That means some of the structures gained in the evolutionary process are preserved in the elite set, while the complete population has guaranteed diversity. The shake operator is usually applied after I_s iterations without improvement in the best overall solution χ^* and, like the ILS perturbation, has strength parameters SK_l , indicating the lower bound, and SK_u , corresponding to the upper bound.

Another common feature is the island model [115]. Inspired by divergent evolution, the island model is the simultaneous evolution of p parallel populations.

The idea is that parallel evolution will lead to different characteristics in the solutions, thereby enabling a more extensive exploration of the solution space. Every g generations, i elite individuals are exchanged among populations, which increases diversity and helps spread novel and interesting structures among the solutions.

Many authors also attempt to introduce interesting structures into the initial population by injecting warm start solutions. This addition is shown to increase the performance and convergence of the method. In general, one or more good solutions are created with other heuristics [1, 4, 18, 54, 64, 100, 116] or mathematical models [17, 98] and then inserted into the initial population. In some cases, the entire initial population can be created using alternative methods, as is done in [120], where the solutions have specific characteristics that are not easily obtained in randomly generated individuals.

One more recent addition to the BRKGA framework is the use of multi-parent crossover [7]. In it, several parents are considered in the mating process, and the choice of inheritance is done using a ranking function. Consider the following example: there are three total parents, denoted as a , b , and c . Of those, both a and b belong to the elite set, while c belongs to the non-elite set. For multi-parent crossover, those parents would be ranked according to their fitness values, so that the first in order would have better fitness than the following ones, and so forth. In this case, considering a minimization problem, observe that $f(a) < f(b) < f(c)$. The ranking of solutions would, thus, be $r(a) = 1$, $r(b) = 2$, and $r(c) = 3$. With these values, the procedure uses a bias function $\Phi(r)$ [12] to calculate the probabilities of each parent being chosen in the mating. The bias function is nonincreasing, so parents with better fitness have a higher chance of being chosen and can belong to several classes, such as exponential, linear, logarithmic, and others.

One last optional procedure is Implicit Path-Relinking [7]. The classical path-linking is an intensification strategy that explores the neighborhood in the path between two distinct solutions [34, 92]. Typically, path-relinking is problem-dependent, due to the exploration of the intermediate solutions. However, the implicit variant leverages the random-key structure of BRKGA to perform the procedure within the random-key solution space, with differences depending on the type *typ* of the decoding process. This procedure works by first selecting two solutions from different populations using method *sel*, which can be the best or randomly chosen among the elite individuals. The selected individuals must have a minimum distance of *md*, which is calculated by either the Hamming (direct IPR) or Kendall Tau (permutation IPR) distances. A percentage *cp%* of the total pairs of elite chromosomes are tested as candidates from IPR, and if no pairs have the minimum distance, then the IPR procedure is not executed. This limitation of candidate pair testing is done to decrease the computational burden of the algorithm. Two other parameters are used in IPR. Parameter *ps%* is the size of the path between the two candidates that is evaluated at each IPR iteration, given as a percentage of the chromosome size. In each test, a block of genes of size *bs* is modified in the base solution to become similar to the one in the guide solution, thus obtaining intermediate solutions that are then explored.

Hybridizing a Random-Key Genetic Algorithm

To increase performance and/or adapt the algorithm to problems with specific characteristics, the authors often incorporate additional methods or frameworks to work in conjunction with BRKGA. This practice is very common for BRKGA, especially when the addition is a local search.

The use of local search (LS) in a genetic framework is viewed as an application of Lamarck's theory of evolution, in which individuals pass on the characteristics they acquire during their lifetime to their descendants [23]. It is particularly common for BRKGA, as it has been shown to significantly increase solution quality and convergence [56]. The use of local search can be within [54, 61] or outside the decoding process, with the latter case applied to all chromosomes in every iteration [14, 100], only the best incumbent solution after evolution is complete [18, 65], or to special chromosomes in some generations [2, 16, 45, 53]. Established methods can also be used as LS, such as Variable Neighborhood Search [21, 60, 96, 104, 105], Clustering Search [19], ILS [99], IGA [1], and INLS [117].

When not performed as a local search, hybridization is typically used to adapt the framework to specific problem characteristics. Some examples are:

- *Multi-objective optimization*: Bierbüße et al. [10], Bolsi et al. [11], Chagas et al. [18], Damm et al. [25], Fontes and Homayouni [31], Fontes et al. [32], Ibarra-Rojas et al. [44], Liu et al. [52], Marques Jr et al. [63], Mikulski and Tomczewski [68], Ramos et al. [88], Raposo et al. [89], Santiyuda et al. [95], and Silva-Soto and Ibarra-Rojas [103]
- *Matheuristics*: Lima et al. [51], Pinacho-Davidson and Blum [84], Pinto et al. [86], and Sun and Zhou [109]
- *Machine learning*: Amaro Júnior et al. [4], Damm et al. [25], Pan et al. [79], Schenekemberg et al. [96], Schenekemberg et al. [97], Xie et al. [116], Xu et al. [117], and Yang et al. [118]
- *Stochastic optimization and scenario generation*: Huang et al. [42], Londe et al. [53], Oliveira et al. [73], Oliveira et al. [74], and Oliveira et al. [75]

Customizing a RKGA

To customize an RKGA, one needs to define how a solution is represented, or encoded, and how the corresponding decoding procedure is done. Also, it is necessary to choose and customize any hybrid methods, if applicable. After this, the algorithm's parameters must be tuned.

Since each solution is represented as a vector of n random keys, it is necessary only to specify a value for n for the encoding. It may represent the number of vertices in a graph, or the total number of jobs and pauses in parallel machines, or the possible locations for facility installation, or the number of parameters to be analyzed, for example.

The decoder is a deterministic procedure that takes as input the chromosome of n random keys and produces as output the solution of the problem alongside the corresponding fitness value. Three types of decoding procedures can be defined as follows: indicator, permutation, and a mix of both. In a permutation decoder, the solution is obtained by sorting the random keys and constructing a feasible solution with the permutation of the indices. Examples of problems that often have permutation-based decoders are Location [53, 65], Cutting and Packing [3, 71], Vehicle Routing [43, 96], Traveling Salesman Problem [4, 18], and Graph Problems [51, 101].

An indicator decoder, in contrast, obtains meaning directly from the values of the random keys. For example, consider that one wishes to assign each point, represented by one random key, to one of the four classes. In this case, the algorithm may consider that the interval $[0, 1)$ is divided into four equal intervals of size 0.25, with the first class being indicated by values in the interval $[0, 0.25)$, the second class by the interval $[0.25, 0.50)$, and so forth. For solution c , if the i -th gene has value $0.25 \leq c[i] < 0.50$, then the corresponding i -th element belongs to the second class. The use of indicator decoders is common in Network Configuration [7, 8, 82, 89], Clustering [64, 72], and Parameter Optimization [22, 29, 45].

Sometimes, a problem has characteristics that make a pure indicator- or permutation-based representation inadequate. When this happens, the authors tend to mix those two, generally by partitioning the chromosome into two or more parts. One example of this practice is done in [113] to locate mobile labs, in which the first genes are sorted to obtain the order of service points, while the last genes indicate an increase of the area of service radius. Another example is [33] for a two-level hub location problem, where the random-key interval is divided by the number of desired hubs. For the genes in each interval, the values are sorted, with the order of the directed graph being indicated by the ordered genes and the first gene corresponding to the hub. In [5], a complex problem related to wireless network design is addressed using a five-part chromosome. The first two sections consist of random keys that indicate the pertaining and parameterization of the solution. The next two sections are used to establish the evaluation order of the deployed equipment and its relation. Lastly, the final section serves as indicators to determine the depths of the network trees.

Tuning an RKGA

Table 1 presents all the parameters that were commented on in the traditional BRKGA and the optional procedures, together with their recommended values from the current literature. One can note that if all procedures are desired, then almost 20 parameters need to be specified to run a BRKGA.

Due to the number of parameters and how much they affect performance, the authors have used many strategies for parameter configuration, either before or during the execution of experiments. When done before, this is called *offline tuning* and is widely used for metaheuristics [26].

Table 1 BRKGA parameters and recommended ranges

Parameter	Recommended value
$ \mathcal{P} $: size of population	$\min\{[100, 500], \lfloor 10 \times n \rfloor\}$
$\mathcal{P}_e\%$: percentage of elite partition of population	[0.1, 0.5]
$\mathcal{P}_m\%$: percentage of mutant partition of population	[0.1, 0.5]
ρ : probability of inheriting key from elite parent	[0.5, 1]
p : number of parallel populations	$\{1, \dots, 3\}$
g : frequency for population interchange	[50, 500]
i : number of elite migrants between parallel populations	$\{1, 2\}$
I_r : iterations without improvement for restart	[200, 500]
I_s : iterations without improvement for shaking	[20, 100]
SK_l : lower bound for shake intensity	[0.1, 0.5]
SK_u : upper bound for shake intensity	[0.5, 0.9]
π_t : total number of parents in multi-parent crossover	[3, 10]
π_e : total number of elite parents in multi-parent crossover	$[1, 7], \pi_e < \pi_t$
$\Phi(r)$: type of bias function for multi-parent crossover	$\{1/\log(r + 1), r^{-1}, r^{-2}, r^{-3}, e^{-r}, 1/\pi_t\}$
sel : individual selection method for path-relinking	$[BESTSOLUTION, RANDOMELITE]$
$cp\%$: percentage of chromosome pairs to be tested for path-relinking	[50, 500]
md : minimum distance between chromosome pairs for path-relinking	[0.0, 0.30]
bs : block size for path-relinking	1.00
$ps\%$: path size for path-relinking	[0.01, 1]
I_{ipr} : frequency of path-relinking	[50, 500]

For BRKGA, the authors have used several offline strategies. One of the simplest methods is a grid search, also known as the full factorial design of experiments. Grid search evaluates all possible combinations of given discrete parameters. However, the complexity of the method increases with each parameter and each candidate range, making it costly to perform if there are many parameters with their own interactions and codependence [30]. Some studies that used grid search are [16, 24, 83, 93].

Other authors utilize parameters suggested by similar groups of problem instances in the literature. However, the performance of a particular algorithm with a specific configuration on some instances is of limited utility, and the generalization of configurations should be done cautiously [47]. The use of literature suggestions is done in [22, 74, 123].

Lastly, one may use an automated method of parameterization. For BRKGA, the authors seem to prefer Iterated F-Race [58], used recently in papers such as [5–8, 53–55, 65].

The parameters may also be tuned during the execution of the algorithm. This process, known as *online parameter control*, falls within the class of machine learning techniques and has been applied to BRKGA in two distinct approaches. In both cases, the parameters are updated at each algorithm generation, aiming to balance intensification and exploitation during evolution. In [4, 20, 99, 117], the authors use the Adaptive BRKGA (A-BRKGA), which updates parameters for population size, elite proportion, mutant proportion, crossover probability, and maximum number of generations at each iteration. It also utilizes two self-adaptive parameters, denoted as α and β , where α indicates the number of members from the elite restricted list that evolve, and β represents the probability of perturbing individuals that are too similar. At each generation, the values of population size, mutant proportion, and α decrease, while elite proportion increases. β and crossover probability are incorporated as genes into the chromosomes and are thus always modified randomly.

The other online approach is called BRKGA-QL [21, 96]. It uses a Q-Learning algorithm [114] to control the same parameters of A-BRKGA in an environment of actions, states, and rewards. Simply put, each possible parameter modification is mapped to a possible reward, and the algorithm chooses one of the actions, with an η probability of selecting the action with the highest reward in terms of intensification. The value of η decays during evolution, so that in later generations, actions that increase diversification tend to be chosen. BRKGA-QL was extended to work with multi-objective problems [25] and had a new reward scheme and diversity methods introduced in [97].

In [112], the author also studies online approaches for BRKGA. It is demonstrated that a random parameter strategy can yield better results compared to more elaborate schemes. This strategy defines upper and lower bounds for parameters $\mathcal{P}$, $\mathcal{P}^e$, $\mathcal{P}^m$, and ρ . Then, in each generation, the parameter values are randomly sampled inside uniform probability distributions between the specified bounds. This approach outperformed the common F-Race-tuned BRKGA in problems with simple decoding strategies.

APIs for BRKGA

To simplify the implementation of BRKGAs, Toso and Resende [110] proposed an *Application Programming Interface (API)* in C++ for BRKGA. The API is efficient and user-friendly. The library is portable and automatically handles several aspects of the BRKGA, including population management and evolutionary dynamics. The API is implemented in C++ and uses an object-oriented architecture. In systems with available *OpenMP* [78], the API enables parallel decoding of random key vectors. The user only needs to implement the decoder and specify the stopping criteria, restart and population exchange mechanisms, as well as the parameters of

the algorithm. The API is open source and can be downloaded from <http://github.com/rfrancotoso/brkgAPI>.

Andrade et al. [7] modernized Toso and Resende [110] leveraging the BRKGA with multi-parent crossover and implicit path-relinking, alongside a multi-population strategy, warm start solution injection, restart, and partial restart features. This framework is also easy to use, as it automatically handles all of those features during the evolutionary process, with the user needing to define only the parameters, decoder, and instance class. The C++ version is available at https://github.com/ceandrade/brkgmp_ipr_cpp, the Python version at https://github.com/ceandrade/brkgmp_ipr_python, and the Julia package at <https://github.com/ceandrade/BrkgMPipr.jl>.

One last API, called BRKGACuda 2.0, is presented by Oliveira et al. [76]. This novel application is optimized to work with parallelism and GPUs, significantly accelerating the algorithm. The API provides all of the classical BRKGA evolutionary operators, with the user only needing to specify the decoder. This C++ algorithm can be found at <https://github.com/discovery-unicamp/brkg-cuda>.

Conclusion

This chapter reviewed random-key genetic algorithms, covering both the unbiased and biased variants. After introducing Bean's algorithm [9], this chapter highlights the minor differences between the RKGGA and the BRKGA that contribute to the improved performance of the biased variant. A model for implementing the BRKGA is described, followed by a discussion of optional procedures within the framework. The use of hybrid methods, as well as the customization of the algorithm regarding decoding processes and tuning methodologies, is also detailed. Finally, this chapter concludes by presenting three APIs for BRKGA that allow for easy implementation of the algorithm.

Cross-References

- [Biased Random-Key Genetic Programming](#)
- [Evolutionary Algorithms](#)
- [Genetic Algorithms](#)
- [Multi-start Methods](#)
- [Restart Strategies](#)
- [Theory of Local Search](#)

Acknowledgments This work was supported by the Brazilian National Council for Scientific and Technological Development (CNPq) under Grant [number 312212/2021-6], Brazilian Coordination for the Improvement of Higher Level Personnel (CAPES) under Grants [numbers 001 and 88887.815411/2023], and the Carlos Chagas Filho Research Support Foundation of the State of Rio de Janeiro (FAPERJ) under Grants [numbers 211.086/2019, 211.389/2019, and 211.588/2021].

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abreu LR, Tavares-Neto RF, Nagano MS (2021) A new efficient biased random key genetic algorithm for open shop scheduling with routing by capacitated single vehicle and makespan minimization. *Eng Appl Artif Intell* 104:104373. <https://doi.org/10.1016/j.engappai.2021.104373>
2. Abreu LR, Athayde Prata B, Gomes AC, Braga-Santos SA, Nagano MS (2022) A novel BRKGA for the customer order scheduling with missing operations to minimize total tardiness. *Swarm Evol Comput* 75:101149. <https://doi.org/10.1016/j.swevo.2022.101149>
3. Amaro Júnior B, Santos MC, Carvalho GN, Araújo LJP, Pinheiro PR (2021) Metaheuristics for the minimum time cut path problem with different cutting and sliding speeds. *Algorithms* 14(11):305. <https://doi.org/10.3390/a14110305>
4. Amaro Júnior B, Carvalho GN, Santos MC, Pinheiro PR, Celedonio JW (2023) Evolutionary algorithms for optimization sequence of cut in the laser cutting path problem. *Appl Sci* 13(18):10133. <https://doi.org/10.3390/app131810133>
5. Andrade CE, Resende MGC, Zhang W, Sinha RK, Reichmann KC, Doverspike RD, Miyazawa FK (2015) A biased random-key genetic algorithm for wireless backhaul network design. *Appl Soft Comput J* 33:150–169. <https://doi.org/10.1016/j.asoc.2015.04.016>
6. Andrade CE, Silva T, Pessoa LS (2019) Minimizing flowtime in a flowshop scheduling problem with a biased random-key genetic algorithm. *Expert Syst Appl* 128:67–80. <https://doi.org/10.1016/j.eswa.2019.03.007>
7. Andrade CE, Toso RF, Gonçalves JF, Resende MGC (2021) The multi-parent biased random-key genetic algorithm with implicit path-relinking and its real-world applications. *Eur J Oper Res* 289(1):17–30. <https://doi.org/10.1016/j.ejor.2019.11.037>
8. Andrade CE, Pessoa LS, Stawiariski S (2024) The physical cell identity assignment problem: a practical optimization approach. *IEEE Trans Evol Comput* 28(2):282–292. <https://doi.org/10.1109/TEVC.2022.3185927>
9. Bean JC (1994) Genetic algorithms and random keys for sequencing and optimization. *ORSA J Comput* 6(2):154–160. <https://doi.org/10.1287/ijoc.6.2.154>
10. Bierbüße J, Mönch L, Biele A (2025) Heuristic approaches for a multi-mode resource availability cost problem in aircraft manufacturing. *Comput Oper Res* 176:106888. <https://doi.org/10.1016/j.cor.2024.106888>
11. Bolsi B, Lima VL, Alves Queiroz TA, Iori M (2022) Heuristic algorithms for integrated workforce allocation and scheduling of perishable products. *Int J Prod Res* 61:1–16. <https://doi.org/10.1080/00207543.2022.2144525>
12. Bresina JL (1996) Heuristic-biased stochastic sampling. In: *Proceedings of the thirteenth national conference on artificial intelligence-volume 1*, pp 271–278
13. Brito J, Fadel A, Semaan G (2020) A genetic algorithm applied to optimal allocation in stratified sampling. *Commun Stat Simul Comput* 51:3714–3732. <https://doi.org/10.1080/03610918.2020.1722832>
14. Brito JA, Semaan G, Fadel A (2022) The effective BRKGA algorithm for the k -medoids clustering problem. *RAIRO-Oper Res* 56(4):3137–3153. <https://doi.org/10.1051/ro/2022141>
15. Buriol LS, Hirsch MJ, Pardalos PM, Querido T, Resende MGC, Ritt M (2010) A biased random-key genetic algorithm for road congestion minimization. *Optim Lett* 4(4):619–633. <https://doi.org/10.1007/s11590-010-0226-6>
16. Carrabs F (2021) A biased random-key genetic algorithm for the set orienteering problem. *Eur J Oper Res* 292(3):830–854. <https://doi.org/10.1016/j.ejor.2020.11.043>
17. Chagas JBC, Toffolo TAM, Souza MJF, Iori M (2020) The double traveling salesman problem with partial last-in-first-out loading constraints. *Int Trans Oper Res* 29(4):2346–2373. <https://doi.org/10.1111/itor.12876>

18. Chagas JBC, Blank J, Wagner M, Souza MJF, Deb K (2021) A non-dominated sorting based customized random-key genetic algorithm for the bi-objective traveling thief problem. *J Heurist* 27(3):267–301. <https://doi.org/10.1007/s10732-020-09457-7>
19. Chaves AA, Lorena LAN, Senne ELF, Resende MGC (2016) Hybrid method with CS and BRKGA applied to the minimization of tool switches problem. *Comput Oper Res* 67:174–183. <https://doi.org/10.1016/j.cor.2015.10.009>
20. Chaves AA, Gonçalves JF, Lorena LAN (2018) Adaptive biased random-key genetic algorithm with local search for the capacitated centered clustering problem. *Comput Ind Eng* 124:331–346. <https://doi.org/10.1016/j.cie.2018.07.031>
21. Chaves AA, Vianna BL, da Silva TT, Schenekemberg CM (2024) A parallel branch-and-cut and an adaptive metaheuristic to solve the family traveling salesman problem. *Expert Syst Appl* 238:121735. <https://doi.org/10.1016/j.eswa.2023.121735>
22. Cicek ZIE, Ozturk ZK (2021) Optimizing the artificial neural network parameters using a biased random key genetic algorithm for time series forecasting. *Appl Soft Comput* 102:107091. <https://doi.org/10.1016/j.asoc.2021.107091>
23. Cotta C, Mathieson L, Moscato P (2018) Memetic algorithms. In: Martí R, Pardalos PM, Resende MGC (eds) *Handbook of heuristics*. Springer International Publishing, Cham, pp 607–638. https://doi.org/10.1007/978-3-319-07124-4_29
24. Damm RB, Resende MGC, Ronconi DP (2016) A biased random key genetic algorithm for the field technician scheduling problem. *Comput Oper Res* 75:49–63. <https://doi.org/10.1016/j.cor.2016.05.003>
25. Damm R, Chaves A, Riveaux J, Ronconi D (2025) Scheduling technicians and tasks through an adaptive multi-objective biased random-key genetic algorithm. *Ann Oper Res* 346(2):945–980. <https://doi.org/10.1007/s10479-024-06325-6>
26. Eiben A, Hinterding R, Michalewicz Z (1999) Parameter control in evolutionary algorithms. *IEEE Trans Evol Comput* 3(2):124–141. <https://doi.org/10.1109/4235.771166>
27. Ericsson M, Resende MGC, Pardalos PM (2002) A genetic algorithm for the weight setting problem in OSPF routing. *J Comb Optim* 6(3):299–333. <https://doi.org/10.1023/A:1014852026591>
28. Fadel AC, Ochi LS, Brito JADM, Semaan GS (2021) Microaggregation heuristic applied to statistical disclosure control. *Inf Sci* 548:37–55. <https://doi.org/10.1016/j.ins.2020.09.069>
29. Falls M, Bernardello R, Castrillo M, Acosta M, Llort J, Galí M (2022) Use of genetic algorithms for ocean model parameter optimisation: a case study using pscs-v2_rc for north atlantic particulate organic carbon. *Geosci Model Develop* 15(14):5713–5737. <https://doi.org/10.5194/gmd-15-5713-2022>
30. Feurer M, Hutter F (2019) *Hyperparameter optimization*. Springer International Publishing, Cham
31. Fontes DBMM, Homayouni SM (2023) A bi-objective multi-population biased random key genetic algorithm for joint scheduling quay cranes and speed adjustable vehicles in container terminals. *Flex Serv Manuf J* 35(1):241–268. <https://doi.org/10.1007/s10696-022-09467-6>
32. Fontes DBMM, Homayouni SM, Fernandes JC (2023) Energy-efficient job shop scheduling problem with transport resources considering speed adjustable resources. *Int J Prod Res* 62:1–24. <https://doi.org/10.1080/00207543.2023.2175172>
33. Freitas CC, Aloise DJ, Da Costa Fontes FF, Santos AC, Da Silva Menezes M (2023) A biased random-key genetic algorithm for the two-level hub location routing problem with directed tours. *OR Spectrum* 45:1–22. <https://doi.org/10.1007/s00291-023-00718-y>
34. Glover F (1997) Tabu search and adaptive memory programming – advances, applications and challenges. In: *Interfaces in computer science and operations research*. Springer, Berlin, pp 1–75. https://doi.org/10.1007/978-1-4615-4102-8_1
35. Gonçalves JF, Almeida JR (2002) A hybrid genetic algorithm for assembly line balancing. *J Heurist* 8(6):629–642. <https://doi.org/10.1023/A:1020377910258>
36. Gonçalves JF, Resende MGC (2011) Biased random-key genetic algorithms for combinatorial optimization. *J Heurist* 17(5):487–525. <https://doi.org/10.1007/s10732-010-9143-1>

37. Gonçalves JF, Wäscher G (2020) A MIP model and a biased random-key genetic algorithm based approach for a two-dimensional cutting problem with defects. *Eur J Oper Res* 286(3):867–882. <https://doi.org/10.1016/j.ejor.2020.04.028>
38. Gonçalves JF, Resende MGC, Toso RF (2014) An experimental comparison of biased and unbiased random-key genetic algorithms. *Pesquisa Oper* 34(2):143–164. <https://doi.org/10.1590/0101-7438.2014.034.02.0143>
39. Holland JH (1975) *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. University Michigan Press, Ann Arbor
40. Huang Y, Santos AC, Duhamel C (2020) Bi-objective methods for road network problems with disruptions and connecting requirements. *J Oper Res Soc* 71(12):1959–1971. <https://doi.org/10.1080/01605682.2019.1639479>
41. Huang Y, Santos AC, Duhamel C (2020) Model and methods to address urban road network problems with disruptions. *Int Trans Oper Res* 27(6):2715–2739. <https://doi.org/10.1111/itor.12641>
42. Huang Z, Huang L, Li D (2024) Co-evolutionary algorithm for two-stage hybrid flow shop scheduling problem with suspension shifts. *Mathematics* 12(16):2575. <https://doi.org/10.3390/math12162575>
43. Ibarra-Rojas OJ, Silva-Soto Y (2021) Vehicle routing problem considering equity of demand satisfaction. *Optim Lett* 15(6):2275–2297. <https://doi.org/10.1007/s11590-021-01704-5>
44. Ibarra-Rojas OJ, Cervantes-Sanmiguel KI, Quintanar-Sanchez DL, Jacobo-Romero YY, Rios-Solis Y (2025) Integration of timetabling to coordinate overlapping lines and vehicle scheduling with interlining. *Transp A Transp Sci* . <https://doi.org/10.1080/23249935.2025.2474640>
45. Japa L, Serqueira M, Mendonça I, Aritsugi M, Bezerra E, González PH (2023) A population-based hybrid approach for hyperparameter optimization of neural networks. *IEEE Access* 11:50752. <https://doi.org/10.1109/ACCESS.2023.3277310>
46. Johnson DS, Breslau L, Diakonikolas I, Duffield N, Gu Y, Hajiaghayi M, Karloff H, Resende MGC, Sen S (2020) Near-optimal disjoint-path facility location through set cover by pairs. *Oper Res* 68(3):896–926. <https://doi.org/10.1287/opre.2019.1956>
47. Karimi-Mamaghan M, Mohammadi M, Meyer P, Karimi-Mamaghan AM, Talbi EG (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. *Eur J Oper Res* 296(2):393–422. <https://doi.org/10.1016/j.ejor.2021.04.032>
48. Kong M, Liu X, Pei J, Cheng H, Pardalos PM (2020) A BRKGA-DE algorithm for parallel-batching scheduling with deterioration and learning effects on parallel machines under preventive maintenance consideration. *Ann Math Artif Intell* 88(1–3):237–267. <https://doi.org/10.1007/s10472-018-9602-1>
49. Kong M, Pei J, Xu J, Liu X, Yu X, Pardalos PM (2020) A robust optimization approach for integrated steel production and batch delivery scheduling with uncertain rolling times and deterioration effect. *Int J Prod Res* 58(17):5132–5154. <https://doi.org/10.1080/00207543.2019.1693659>
50. Kummer AF, Araújo OCB, Buriol LS, Resende MGC (2022) A biased random-key genetic algorithm for the home health care problem. *Int Trans Oper Res* 31:1–31. <https://doi.org/10.1111/itor.13221>
51. Lima A, Aquino ALL, Nogueira B, Pinheiro RGS (2022) A matheuristic approach for the minimum broadcast time problem using a biased random-key genetic algorithm. *Int Trans Oper Res* 31:246–273. <https://doi.org/10.1111/itor.13146>
52. Liu C, Han Y, Wang Y, Li J, Liu Y (2024) Multi-population coevolutionary algorithm for a green multi-objective flexible job shop scheduling problem with automated guided vehicles and variable processing speed constraints. *Swarm Evol Comput* 91:101774. <https://doi.org/10.1016/j.swevo.2024.101774>
53. Londe MA, Andrade CE, Pessoa LS (2021) An evolutionary approach for the p-next center problem. *Expert Syst Appl* 175:114728. <https://doi.org/10.1016/j.eswa.2021.114728>

54. Londe MA, Andrade CE, Pessoa LS (2022) Exact and heuristic approaches for the root sequence index allocation problem. *Appl Soft Comput* 130:109634. <https://doi.org/10.1016/j.asoc.2022.109634>
55. Londe MA, Andrade CE, Pessoa LS (2025) A multi-stage approach for root sequence index allocation. *Eur J Oper Res* <https://doi.org/10.1016/j.ejor.2025.05.015>. Article in Press
56. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Biased random-key genetic algorithms: a review. *Eur J Oper Res* 321:1–22. <https://doi.org/10.1016/j.ejor.2024.03.030>
57. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Early years of biased random-key genetic algorithms: a systematic review. *J Global Optim* <https://doi.org/10.1007/s10898-024-01446-5>. To Appear
58. López-Ibáñez M, Dubois-Lacoste J, Cáceres LP, Birattari M, Stützle T (2016) The irace package: iterated racing for automatic algorithm configuration. *Oper Res Perspect* 3:43–58. <https://doi.org/10.1016/j.orp.2016.09.002>
59. Lourenço HR, Martin OCM, Stützle T (2003) Iterated local search. In: *Handbook of metaheuristics*. Springer, Berlin, pp 320–353. https://doi.org/10.1007/0-306-48056-5_11
60. Lu Z, Hu K, Ng TS (2023) Improving additive manufacturing production planning: a sub-second pixel-based packing algorithm. *Comput Ind Eng* 181:109318. <https://doi.org/10.1016/j.cie.2023.109318>
61. Maecker S, Shen L, Mönch L (2023) Unrelated parallel machine scheduling with eligibility constraints and delivery times to minimize total weighted tardiness. *Comput Oper Res* 149:105999. <https://doi.org/10.1016/j.cor.2022.105999>
62. Mandal SK, de Queiroz TA, Miyazawa FK (2024) A biased random-key genetic algorithm for the 2d strip packing problem with order and stability constraints. *Pesquisa Operacional* 44. <https://doi.org/10.1590/0101-7438.2023.043.00284365>
63. Marques Jr EL, Coelho VN, Coelho IM, Satoru Ochi L, Maculan N, Mladenović N, Coelho BN (2023) A two-phase multi-objective metaheuristic for a green UAV grid routing problem. *Optim Lett* 17:1–24. <https://doi.org/10.1007/s11590-023-02013-9>
64. Martarelli NJ, Nagano MS (2020) Unsupervised feature selection based on bio-inspired approaches. *Swarm Evol Comput* 52:100618. <https://doi.org/10.1016/j.swevo.2019.100618>
65. Mauri GR, Biajoli FL, Rabello RL, Chaves AA, Ribeiro GM, Lorena LAN (2021) Hybrid metaheuristics to solve a multiproduct two-stage capacitated facility location problem. *Int Trans Oper Res* 28(6):3069–3093. <https://doi.org/10.1111/itor.12930>
66. Melo RA, Ribeiro CC, Riveaux JA (2023) A biased random-key genetic algorithm for the minimum quasi-clique partitioning problem. *Ann Oper Res* 1–33. <https://doi.org/10.1007/s10479-023-05609-7>
67. Mendoza-Andrade N, Ruiz-y-Ruiz E, Rodriguez-Romo S (2025) Solution algorithms for the capacitated location tree problem with interconnections. *Algorithms* 18(1):50. <https://doi.org/10.3390/a18010050>
68. Mikulski S, Tomczewski A (2021) Use of energy storage to reduce transmission losses in meshed power distribution networks. *Energies* 14(21):7304. <https://doi.org/10.3390/en14217304>
69. Morgan SJ, McGrath CN, Weck OL (2023) Optimization of multispacecraft maneuvers for mobile target tracking from low earth orbit. *JSpacecraft Rockets* 60(2):581–590. <https://doi.org/10.2514/1.A35457>
70. Ochoa G, Malan K, Blum C (2021) Search trajectory networks: a tool for analysing and visualising the behaviour of metaheuristics. *Appl Soft Comput* 109:107492. <https://doi.org/10.1016/j.asoc.2021.107492>
71. Oliveira EV, Romero R (2022) Improved biased random key genetic algorithm for the two-dimensional non-guillotine cutting problem. *Pesquisa Oper* 42:1–24. <https://doi.org/10.1590/0101-7438.2022.042.00260406>
72. Oliveira RMD, Chaves AA, Lorena LAN (2017) A comparison of two hybrid methods for constrained clustering problems. *Appl Soft Comput J* 54:256–266. <https://doi.org/10.1016/j.asoc.2017.01.023>

73. Oliveira BB, Carravilla MA, Oliveira JF, Costa AM (2019) A co-evolutionary matheuristic for the car rental capacity-pricing stochastic problem. *Eur J Oper Res* 276(2):637–655. <https://doi.org/10.1016/j.ejor.2019.01.015>
74. Oliveira BB, Carravilla MA, Oliveira JF, Resende MGC (2021) A C++ application programming interface for co-evolutionary biased random-key genetic algorithms for solution and scenario generation. *Optim Methods Softw* 37(3):1065–1086. <https://doi.org/10.1080/10556788.2021.1884250>
75. Oliveira BB, Carravilla MA, Oliveira JF (2022) A diversity-based genetic algorithm for scenario generation. *Eur J Oper Res* 299(3):1128–1141. <https://doi.org/10.1016/j.ejor.2021.09.047>
76. Oliveira BA, Xavier EC, Borin E (2024) BrkgaCuda 2.0: a framework for fast biased random-key genetic algorithms on GPUs. *Soft Comput* 28(21):12689–12704. <https://doi.org/10.1007/s00500-024-10336-7>
77. Oliveira JaLA, Aquino AL, Pinheiro RG, Nogueira B (2024) Optimizing public transport system using biased random-key genetic algorithm. *Appl Soft Comput* 158:111578. <https://doi.org/10.1016/j.asoc.2024.111578>
78. OpenMP (2013) <http://openmp.org/wp/>. Last Visited on May 11, 2013
79. Pan Y, Zhang L, Yan Z, Lwin M, Skibniewski M (2021) Discovering optimal strategies for mitigating COVID-19 spread using machine learning: experience from Asia. *Sustain Cities Soc* 75:103254. <https://doi.org/10.1016/j.scs.2021.103254>
80. Pandey HM, Chaudhary A, Mehrotra D (2014) A comparative review of approaches to prevent premature convergence in GA. *Appl Soft Comput* 24:1047–1077. <https://doi.org/10.1016/j.asoc.2014.08.025>
81. Pastore T, Menna C, Asprone D (2022) Bézier-based biased random-key genetic algorithm to address printability constraints in the topology optimization of concrete structures. *Struct Multidisc Optim* 65(2):64. <https://doi.org/10.1007/s00158-021-03119-3>
82. Pedrola O, Careglio D, Klinkowski M, Velasco L, Bergman K, Solé-Pareta J (2013) Metaheuristic hybridizations for the regenerator placement and dimensioning problem in sub-wavelength switching optical networks. *Eur J Oper Res* 224(3):614–624. <https://doi.org/10.1016/j.ejor.2012.08.011>
83. Pessoa LS, Santos AC, Resende MGC (2017) A biased random-key genetic algorithm for the tree of hubs location problem. *Optim Lett* 11(7):1371–1384. <https://doi.org/10.1007/s11590-016-1082-9>
84. Pinacho-Davidson P, Blum C (2020) Barrakuda: a hybrid evolutionary algorithm for minimum capacitated dominating set problem. *Mathematics* 8(11):1–26. <https://doi.org/10.3390/math8111858>
85. Pinto BQ, Ribeiro CC, Rosseti I, Noronha TF (2020) A biased random-key genetic algorithm for routing and wavelength assignment under a sliding scheduled traffic model. *J Global Optim* 77(4):949–973. <https://doi.org/10.1007/s10898-020-00877-0>
86. Pinto BQ, Ribeiro CC, Riveaux JA, Rosseti I (2021) A BRKGA-based matheuristic for the maximum quasi-clique problem with an exact local search strategy. *RAIRO-Oper Res* 55:S741–S763. <https://doi.org/10.1051/ro/2020003>
87. Queiroga E, Pinheiro RGS, Christ Q, Subramanian A, Pessoa AA (2021) Iterated local search for single machine total weighted tardiness batch scheduling. *J Heurist* 27(3):353–438. <https://doi.org/10.1007/s10732-020-09461-x>
88. Ramos S, Roque LA, Gomes A, Soares J, Rolle JC, Vale Z (2025) Multi-objective model for residential energy management in context of individual self-consumption. *Math Comput Simul* 231:120–127. <https://doi.org/10.1016/j.matcom.2024.11.017>
89. Raposo AAM, Rodrigues AB, da Guia da Silva M (2020) Robust meter placement for state estimation considering distribution network reconfiguration for annual energy loss reduction. *Electric Power Syst Res* 182:106233. <https://doi.org/10.1016/j.epsr.2020.106233>
90. Reixach J, Blum C, Djukanovic M, Raidl GR (2025) A biased random key genetic algorithm for solving the longest common square subsequence problem. *IEEE Trans Evol Comput* 29(2):390–403. <https://doi.org/10.1109/TEVC.2024.3413150>

91. Resende MGC, Ribeiro CC (2011) Restart strategies for GRASP with path-relinking heuristics. *Optim Lett* 5:467–478
92. Ribeiro CC, Resende MGC (2012) Path-relinking intensification methods for stochastic local search algorithms. *J Heurist* 18(2):193–214. <https://doi.org/10.1007/s10732-011-9167-1>
93. Rochman AN, Prasetyo H, Nugroho MT (2017) Biased random key genetic algorithm with insertion and gender selection for capacitated vehicle routing problem with time windows. *AIP Confer Proc* 1855(1):020025. <https://doi.org/10.1063/1.4985470>
94. Rocholl J, Mönch L (2021) Decomposition heuristics for parallel-machine multiple orders per job scheduling problems with a common due date. *J Oper Res Soc* 72(8):1737–1753. <https://doi.org/10.1080/01605682.2019.1640589>
95. Santiyuda G, Wardoyo R, Pulungan R (2024) Solving biobjective traveling thief problems with multiobjective reinforcement learning. *Appl Soft Comput* 161:111751. <https://doi.org/10.1016/j.asoc.2024.111751>
96. Schenekemberg CM, Chaves AA, Coelho LC, Guimarães TA, Avelino GG (2022) The dial-a-ride problem with private fleet and common carrier. *Comput Oper Res* 147:105933. <https://doi.org/10.1016/j.cor.2022.105933>
97. Schenekemberg CM, Chaves AA, Guimarães TA, Coelho LC (2024) Hybrid metaheuristic for the dial-a-ride problem with private fleet and common carrier integrated with public transportation. *Ann Oper Res* <https://doi.org/10.1007/s10479-024-06136-9>
98. Schuetz MJA, Brubaker JK, Montagu H, van Dijk Y, Klepsch J, Ross P, Luckow A, Resende MGC, Katzgraber HG (2022) Optimization of robot-trajectory planning with nature-inspired and hybrid quantum algorithms. *Phys Rev Appl* 18(5):054045. <https://doi.org/10.1103/PhysRevApplied.18.054045>
99. Silva TT, Chaves AA, Yanasse HH, Luna HPL (2019) The multicommodity traveling salesman problem with priority prizes: a mathematical model and metaheuristics. *Comput Appl Math* 38(4):188. <https://doi.org/10.1007/s40314-019-0976-4>
100. Silva G, Leite A, Ospina R, Leiva V, Figueroa-Zúñiga J, Castro C (2023) Biased random-key genetic algorithm with local search applied to the maximum diversity problem. *Mathematics* 11(14):3072. <https://doi.org/10.3390/math11143072>
101. Silva SE, Ribeiro CC, dos Santos Souza U (2023) A biased random-key genetic algorithm for the chordal completion problem. *RAIRO-Oper Res* 57(3):1559–1578. <https://doi.org/10.1051/ro/2023081>
102. Silva MC, Melo RA, Resende MG, Santos MC, Toso RF (2024) Obtaining the Grundy chromatic number: how bad can my greedy heuristic coloring be? *Comput Oper Res* 168:106703. <https://doi.org/10.1016/j.cor.2024.106703>
103. Silva-Soto YI, Ibarra-Rojas OJ (2021) Timetabling with flexible frequencies to synchronise groups of bus lines at common stops. *Transp A Transp Sci* 17(4):978–1001. <https://doi.org/10.1080/23249935.2020.1822952>
104. Soares LCR, Carvalho MAM (2020) Biased random-key genetic algorithm for scheduling identical parallel machines with tooling constraints. *Eur J Oper Res* 285(3):955–964. <https://doi.org/10.1016/j.ejor.2020.02.047>
105. Soares LC, Carvalho MA (2022) Application of a hybrid evolutionary algorithm to resource-constrained parallel machine scheduling with setup times. *Comput Oper Res* 139:105637. <https://doi.org/10.1016/j.cor.2021.105637>
106. Souza Queiroz LRD, Andretta M (2020) Two effective methods for the irregular knapsack problem. *Appl Soft Comput J* 95:106485. <https://doi.org/10.1016/j.asoc.2020.106485>
107. Spears WM, De Jong KA (1991) An analysis of multi-point crossover. In: *Foundations of genetic algorithms*, vol 1. Elsevier, Amsterdam, pp 301–315. <https://doi.org/10.1016/B978-0-08-050684-5.50022-7>
108. Spears WM, De Jong KA (1991) On the virtues of parameterized uniform crossover. In: *Proceedings of the fourth international conference on genetic algorithms*, pp 230–236
109. Sun B, Zhou Y (2022) Bayesian network structure learning with improved genetic algorithm. *Int J Intell Syst* 37(9):6023–6047. <https://doi.org/10.1002/int.22833>

110. Toso RF, Resende MGC (2015) A C++ application programming interface for biased random-key genetic algorithms. *Optim Methods Softw* 30(1):81–93. <https://doi.org/10.1080/10556788.2014.890197>
111. Usberti FL, González JFV, Cavellucci C (2024) A quadratic formulation for the optimal allocation of fault indicators. *Electric Power Syst Res* 231:110352. <https://doi.org/10.1016/j.eprsr.2024.110352>
112. Vilela LZAR (2022) Strategies for parameter control in the biased random-key genetic algorithm. Master's thesis, Pontifical Catholic University of Rio de Janeiro. <https://doi.org/10.17771/PUCRio.acad.61145>
113. Villicaña-Cervantes D, Ibarra-Rojas OJ (2022) Accessible location of mobile labs for COVID-19 testing. *Health Care Manag Sci* 27:1–19. <https://doi.org/10.1007/s10729-022-09614-3>
114. Watkins CJ, Dayan P (1992) Q-learning. *Mach Learn* 8:279–292. <https://doi.org/10.1007/BF00992698>
115. Whitley D, Rana S, Heckendorn RB (1999) The island model genetic algorithm: on separability, population size and convergence. *J Comput Inf Technol* 7(1):33–47
116. Xie Y, Sheng Y, Qiu M, Gui F (2022) An adaptive decoding biased random key genetic algorithm for cloud workflow scheduling. *Eng Appl Artif Intell* 112:104879. <https://doi.org/10.1016/j.engappai.2022.104879>
117. Xu Y, Guo P, Zeng Y (2022) An iterative neighborhood local search algorithm for capacitated centered clustering problem. *IEEE Access* 10:34497–34510. <https://doi.org/10.1109/ACCESS.2022.3162692>
118. Yang F, Li C, Wang F, Yang Z, Gao K (2025) Scheduling two-stage healthcare appointment systems via a knowledge-based biased random-key genetic algorithm. *Swarm Evol Comput* 94:101864. <https://doi.org/10.1016/j.swevo.2025.101864>
119. Yu X, Qian J, Zhang Y, Kong M (2023) Supply chain scheduling method for the coordination of agile production and port delivery operation. *Mathematics* 11(15):3276. <https://doi.org/10.3390/math11153276>
120. Zhang X, Shan M, Zeng J (2022) Parallel batch processing machine scheduling under two-dimensional bin-packing constraints. *IEEE Trans Reliab* 72:1265. <https://doi.org/10.1109/TR.2022.3201333>
121. Zhao W, Liu Y, Wang Y, Yue X (2023) Distributed spot welding task allocation and sequential planning for multi-station multi-robot coordinate assembly processes. *Int J Adv Manuf Technol* 127(11–12):5233–5251. <https://doi.org/10.1007/s00170-023-11750-1>
122. Zhou G, Li D, Bian J, Zhang Y (2024) Airfreight forwarder's shipment planning: shipment consolidation and containerization. *Comput Oper Res* 161:106443. <https://doi.org/10.1016/j.cor.2023.106443>
123. Zudio A, da Silva Costa DH, Masquio BP, Coelho IM, Pinto PED (2018) BRKGA/VND hybrid algorithm for the classic three-dimensional bin packing problem. *Electron Notes Discrete Math* 66:175–182. <https://doi.org/10.1016/j.endm.2018.03.023>